

RootCause-SLM

Block-level anomaly detection on HDFS logs,
and an experiment on explaining anomalies with a small language model

Yifan Wang

Télécom Paris

<https://github.com/NafiyTP/RootCause-SLM>

Abstract

This report studies anomaly detection on HDFS logs, and then asks whether a small language model running locally can explain the anomalies it finds. Raw lines are grouped into block sessions and each block is classified from counts of message templates. On the full HDFS_v1 log (11.2M lines, 575k blocks, chronological split) a logistic regression reaches an F1 of 0.973, against 0.398 for a log-level rule and 0.090 for PCA, and an end-to-end run on 3,000 consecutive test blocks flags exactly the 15 anomalies. The detector needs complete sessions and stable templates: on a truncated extract its F1 drops to 0.27, and on BGL, a less regular system, to 0.676, below a log-level rule (0.846). For the explanation experiment, Qwen2.5-1.5B-Instruct is fine-tuned with LoRA on explanations written by Llama 3.3-70B. The student reproduces its teacher better than every baseline (ROUGE-L 0.506 on the reasoning, against 0.456 for template retrieval) and, with batching, costs about seven times less per explanation. An audit against the raw logs shows the limit of the setup: on 250 anomalous test lines, the fine-tuned model never names what is actually wrong with the block, and the teacher does so once, because the evidence lies in other lines of the block. A second version gives the same small model a summary of the whole block instead of one line, with new annotations from GPT-OSS 120B and an evaluation against the raw log: the fine-tuned 1.5B model then names the right cause for 63% of the anomalous test blocks, close to its teacher (67%), and for 100% of the three most common failures, where the one-line version scored between 0 and 27%. It inherits its teacher’s blind spots, which makes the quality of the annotations, not the size of the student, the next limit.

1 Introduction

Logs are often the first place an operator looks when a distributed system misbehaves, and they are far too large to read by hand. On HDFS, a block id ties every line to a session and a few dozen message types describe almost every event, which makes it the standard benchmark for log anomaly detection [1, 2, 3, 4]. The main part of this project is a block-level detector evaluated the way it would be used: on the full log, trained on the past and tested on the future, compared with the simple rule an operator would write by hand, and run on a second, less regular system to see where it breaks.

After an alert, the next question is what went wrong. Large language models can write such explanations, but sending production logs to an external API is often not allowed. The second part is an experiment on that question: can a small model, small enough to run on a single cheap GPU, learn to explain anomalies from a large one, and how can one tell whether its explanations are right?

Contributions.

- A block-level detector evaluated on the full HDFS_v1 log with a chronological split (F1 0.973), compared with a log-level rule and PCA, with its two failure modes measured: truncated sessions and an unstable template vocabulary (BGL).

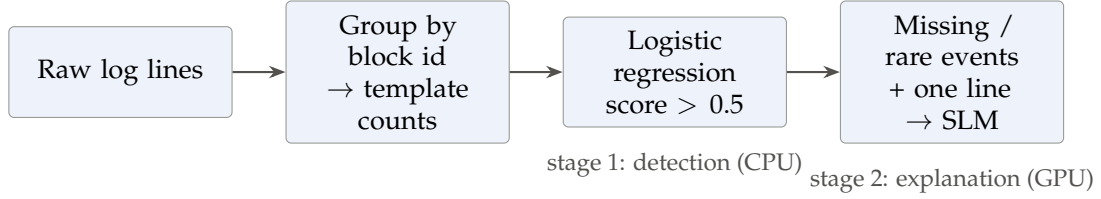


Figure 1: The pipeline. Detection runs on the whole block; the language model sees a single line.

- A teacher-student explanation model (Llama 3.3-70B → Qwen2.5-1.5B with LoRA), evaluated against five baselines, with its inference cost measured on a T4.
- An audit showing that neither the teacher nor the student identifies the block-level evidence, with an analysis of why, and a zero-cost alternative (missing events) that does.
- A second version with block-level input, evaluated against the raw log, in which the same 1.5B student gets close to a 120B teacher on finding the cause.

2 Data

HDFS_v1. The Loghub release of the HDFS logs collected by Xu et al. from Hadoop map-reduce jobs on more than 200 Amazon EC2 nodes in 2008 [1, 4]: 11,175,629 lines and 575,061 blocks, of which 2.9% are labelled anomalous by Hadoop experts. Labels are per block. A line belongs to every block whose id (`blk_-\d+`) it mentions.

BGL. Logs of the Blue Gene/L supercomputer at Lawrence Livermore National Laboratory [5], 4,713,493 lines. There is no session id, and every line carries its own label: – for normal, otherwise an alert category.

Explanation dataset. For the explanation model, each line gets the label of its block, and Llama 3.3-70B (through the Groq API) writes two fields in French: a short *cause* and a three-step *raisonnement*. The teacher never decides the label. The training set has 1,999 lines (1,930 normal, 69 anomalous, split 85/15 into train and validation); the anomalous lines cover 29 different causes and the whole set only 15 message templates. The test set has 527 lines (277 normal, 250 anomalous) taken from blocks that never appear in training, with anomalies oversampled so that the metrics say something about them.

3 Method

3.1 Templates

Each line is reduced to a template: the date, time and process id are dropped, then block ids, IP addresses (with an optional port), paths and remaining numbers are replaced by `<BLK>`, `<IP>`, `<PATH>` and `<N>`, in that order. On HDFS this yields 54 templates over the training period. For BGL the masking also replaces hexadecimal values and the message is prefixed with the component and the level. No learned parser such as Drain [6] is used.

3.2 Block-level detection

A block becomes a vector $x \in \mathbb{N}^{V+1}$ of template counts, with one extra column `<UNK>` that collects the templates absent from training (a new kind of message is exactly what one wants to notice, so it is kept rather than dropped). Blocks are sorted by their first line and split chronologically: the

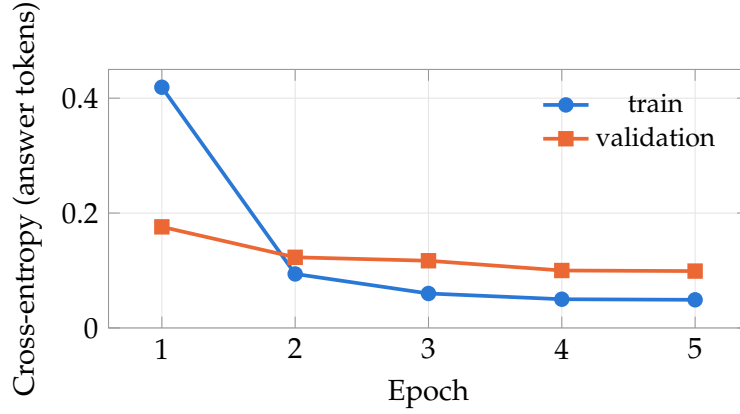


Figure 2: Training curves. Validation perplexity is already 1.19 after one epoch and ends at 1.10: the targets are short and very repetitive.

first 80% for training, the last 20% for testing, so the model never sees the future. Three detectors are compared.

- **Log-level rule:** a block is anomalous if one of its lines is not INFO (on BGL: FATAL, FAILURE, SEVERE or ERROR). This is what one does without learning.
- **PCA [1]:** fit on standardised $\log(1 + x)$, keep the components that explain 95% of the variance, and score a block by its squared reconstruction error. The threshold is the 97th percentile of the training scores; no label is used.
- **Logistic regression** on $\log(1 + x)$ with balanced class weights (anomalies are about 3% of the blocks), threshold 0.5.

On BGL, a session is a 60-minute window over the whole machine, anomalous if it contains at least one alert line.

3.3 Explanation model

The student is Qwen2.5-1.5B-Instruct [7]. The prompt is the ChatML system message below, then the line (truncated to 300 characters) and its label; the target is the teacher’s JSON. The loss is computed only on the answer tokens (prompt positions set to -100). LoRA [8] adapters of rank 16 ($\alpha = 32$, dropout 0.05) are trained on the four attention projections, 4.36M parameters or 0.28% of the model. Because anomalies are 3.5% of the training lines, a weighted random sampler draws both classes about equally often. Training uses AdamW with learning rate $2 \cdot 10^{-4}$, 5% linear warm-up then cosine decay, an effective batch size of 16 and 5 epochs on a Colab T4 (Figure 2).

Tu es un expert en systèmes distribués Hadoop/HDFS. Étant donné un log et son label (Normal ou Anomaly), tu fournis une cause technique précise et un raisonnement en 3 étapes.

3.4 From a flagged block to an explanation

The model was trained on single lines, so the pipeline has to choose one. For every flagged block it takes the line whose template pushes the score up the most: a template never seen in training if there is one, otherwise the one with the largest coefficient. That line is sent with the label *Anomaly*. Independently of the model, the pipeline also lists the *missing events* (templates that at least 90% of the normal training blocks contain but this block does not) and the *rare events* (present in the block, seen in fewer than 5% of normal blocks), and turns them into one English sentence with a short ordered list of rules, for example: *The block was allocated but never stored: the events a normal write ends with (addStoredBlock, PacketResponder terminating) never appear. The block also logged java.io.IOException: Could not read from stream.*

Table 1: Detection on the full HDFS_v1 log. Test set: the last 115,013 blocks (1.5% anomalous).

Detector	Precision	Recall	F1	Flagged
WARN/ERROR rule	1.000	0.248	0.398	417
PCA (unsupervised)	0.379	0.051	0.090	227
Logistic regression	0.955	0.992	0.973	1,744

Table 2: The same detectors on BGL, 60-minute windows. Test set: the last 724 windows (18.6% anomalous). 13,603 templates in training; 412 test windows (57%) contain at least one unseen template.

Detector	Precision	Recall	F1	Flagged
Level rule (FATAL/FAILURE/SEVERE/ERROR)	0.734	1.000	0.846	184
PCA (unsupervised)	0.203	0.541	0.295	359
Logistic regression	0.972	0.518	0.676	72

4 Results

4.1 Detection on HDFS

Table 1 gives the results. The rule never raises a false alarm but finds only a quarter of the anomalies: three quarters of the anomalous blocks contain only INFO lines. The logistic regression finds almost all of them. Its largest weights go to the templates one would expect: a redundant `addStoredBlock`, an `addStoredBlock` for a block that belongs to no file, a failed delete (*BlockInfo not found in volumeMap*) and a replication time-out. Parsing, features and prediction together run at about 115k lines per second on one CPU core.

PCA ranks the blocks well (ROC AUC 0.98 on the test set) but its threshold does not transfer: it is fixed on the training period, where 3.3% of the blocks are anomalous, against 1.5% in the test period, and the scores shift between the two. With the threshold fixed on training scores it flags only 0.2% of the test blocks. A usable threshold needs labels or a known anomaly rate.

The detector also needs complete sessions. On an extract made of the first 104,815 lines, the same logistic regression only reaches F1 0.27: the most recent test blocks are cut at the end of the extract, look exactly like writes that never completed, and are flagged. On a stream, a block has to be closed before it can be scored.

4.2 Generalisation to BGL

On BGL the ranking flips (Table 2, Figure 3). The logistic regression is still very precise but misses half of the anomalous windows, and the level rule wins. The template counts explain it: the masking written for HDFS leaves 13,603 templates on BGL, and 57% of the test windows contain at least one template never seen in training. All of them land in the single `<UNK>` column, so a new kind of failure looks almost like a new kind of normal message. Counting templates only works when templates are stable; on a less regular system it needs a real parser and a model that can place unseen templates.

4.3 Explanations compared with the teacher

Table 3 compares the fine-tuned model with three non-neural baselines and with the base model prompted zero-shot and few-shot. ROUGE-L [9] and BERTScore [10] are computed against the teacher. Template retrieval, which copies the annotation of a training line with the same template and label, is the baseline that matters. The fine-tuned model beats it by 0.050 ROUGE-L on the

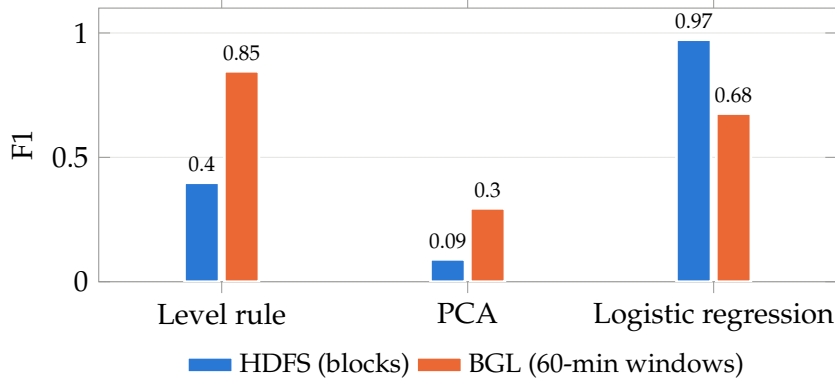


Figure 3: F1 of each detector on the two systems. The ranking between the rule and the learned model flips.

Table 3: Explanation quality on the 527 test lines, measured against the teacher’s annotations. Greedy decoding; the fine-tuned model gets exactly its training prompt.

System	Valid JSON	Cause exact	ROUGE-L cause	ROUGE-L reasoning	BERTScore reasoning
Most frequent cause	100%	0.461	0.594	0.347	0.797
Majority cause per label	100%	0.461	0.589	0.356	0.783
Template retrieval	100%	0.548	0.860	0.456	0.825
Qwen2.5-1.5B zero-shot	6.5%	0.000	0.008	0.017	0.048
Qwen2.5-1.5B few-shot (3 ex.)	100%	0.000	0.393	0.292	0.778
Qwen2.5-1.5B + LoRA	100%	0.617	0.874	0.506	0.836

reasoning (paired bootstrap 95% CI [0.038, 0.064]) and doubles the exact cause match on anomalies (0.284 against 0.140); on the reasoning of anomalous lines the two are tied (0.455 against 0.461). Zero-shot, the base model mostly fails on format (code fences, broken strings, running out of tokens); three examples fix the format, but it then paraphrases the causes. So, measured against its teacher, the distillation works.

4.4 Audit: do the explanations find the real cause?

The scores above say how close each system gets to the teacher, not whether the cause is right. To check that, the full block of every anomalous test line was rebuilt from `HDFS.log` and its block-level evidence extracted with the detector’s statistics: if the events that normal blocks end with (including the `addStoredBlock` that confirms a replica was stored) are missing, the write never completed; otherwise the rare events it contains (seen in fewer than 5% of normal blocks) name the problem. Then each explanation was checked for a mention of that evidence with keywords, and 50 of them (25 anomalous, 25 normal, fixed seed) were read by hand to make sure the keyword check misses nothing.

Figure 4 shows the result. The evidence is always somewhere in the block, and almost never in the line the model sees. Both models tell one of two stories, chosen by the template of that line: an `allocateBlock` line becomes *Erreur d’allocation de bloc* (183 of 250 lines) and a `Receiving block` line becomes *Connexion non autorisée* (65 lines). Neither is supported by the logs: the allocation succeeded, and nothing in the block points to an unauthorised connection. On normal lines the explanations are fine, since they only describe the event. The fine-tuned model uses 8 distinct causes on the test set, the teacher 19.

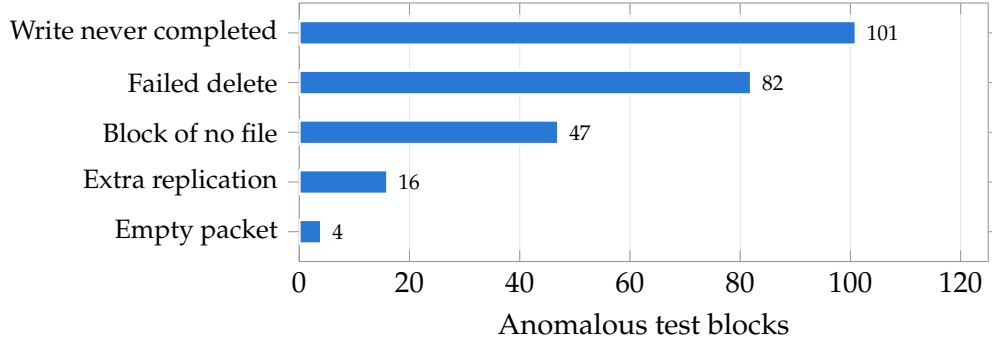


Figure 4: What is actually wrong with the 250 anomalous test blocks. The teacher mentions this evidence in 1 explanation out of 250, the fine-tuned model in none.

Table 4: v2 explanations on 123 test blocks, scored against the raw log. For anomalous blocks, correct + wrong or vague + missed = 100%. “Invents”: share of normal blocks for which the system claims a problem. v1 is told the block is anomalous, so its normal-block columns are empty.

System	Valid JSON	Correct cause	Wrong or vague	Missed	Invents on normal random	re-repl.
Rules (pipeline sentence)	100%	100%	0%	0%	60%	100%
Teacher, GPT-OSS 120B	100%	67%	13%	19%	40%	0%
v1 student (one line)	99%	39%	61%	0%	–	–
Qwen2.5-1.5B, no fine-tuning	27%	4%	0%	96%	0%	0%
v2 student (block summary)	100%	63%	24%	13%	40%	0%

4.5 v2: explaining from the whole block

The audit points at the input, so the second version changes the input and keeps the model. The student now receives a summary of the whole block: every event type with its count, in order of first appearance, the raw WARN and exception lines, and the five events a normal write always has. It is not told the label and answers in JSON with `anomalous`, `cause`, `evidence` (events from the summary, `missing`: for an absent usual event) and a `short explanation`. The data use the detector’s chronological split: 409 training blocks and 123 test blocks (83 anomalous, sampled per kind of failure, and 40 normal, half of them re-replication blocks, the pattern behind all of the detector’s false alarms). The teacher is GPT-OSS 120B on Groq (temperature 0, one block per call); the student is the same Qwen2.5-1.5B with the same LoRA recipe, trained with batch size 1, 16 accumulation steps and gradient checkpointing to fit the longer inputs on a T4 (validation perplexity 1.28 to 1.16).

Every system is scored against the raw log. The gold cause comes from the pipeline rules (Section 3.4); an answer is matched to a cause mainly through the events it cites, with a few keywords as a complement. I checked this matching by hand on all 123 teacher answers: the 57 counted correct are correct and the 26 counted wrong are real mistakes.

Tables 4 and 5 give the results. With the whole block as input, the 1.5B student names the right cause for 63% of the anomalous blocks, close to its 120B teacher (67%), and 98% of the evidence it cites is really in its input. On the three most common failures it reaches 100%, where v1 scored 0%, 13% and 27%. Without fine-tuning the same model produces valid JSON only 27% of the time. The student also inherits its teacher’s blind spots: both treat normal block deletions as a problem (all 8 problems v2 invents on normal blocks are “block deletion”) and both miss empty packets, which are INFO lines with no error. Its 0% on normal re-replication blocks is not discrimination: it never calls a re-replication a problem, including the anomalous ones. v1 does well on re-replication and empty packets only because, for those blocks, the line it receives is the

Table 5: Correct cause per kind of failure (v2 test set).

Failure	n	Teacher	v1	v2
Write never completed	15	100%	0%	100%
Failed delete	15	100%	13%	100%
Redundant addStoredBlock	15	100%	27%	100%
Block of no file	5	100%	20%	60%
Replication time-out	3	100%	100%	100%
Empty packet	15	0%	47%	7%
Re-replication	15	20%	100%	0%

Table 6: Inference on a Tesla T4 (fp16, LoRA merged), 64 test lines, average request 161 tokens in and 92 out. GPU at \$0.35 per hour; Groq at \$0.59 / \$0.79 per million input / output tokens. The teacher cost is estimated from token counts of the same prompts and answers.

Setup	s / line	lines / s	tokens / s	USD / 1k lines
Qwen2.5-1.5B + LoRA, batch 1	2.918	0.34	30	0.284
Qwen2.5-1.5B + LoRA, batch 16	0.251	3.99	344	0.024
Llama 3.3-70B on Groq (estimate)				0.168

anomalous event itself.

4.6 End to end

The pipeline was run on 3,000 consecutive blocks from the detection test period, with all their lines (38,957 lines, 15 anomalous blocks). It flagged exactly the 15 anomalies, with no false alarm, in 0.4 seconds. For 12 of them the missing events describe the failure: the block was allocated and the transfer started, but `addStoredBlock` and `PacketResponder` terminating never appear. For those 12 the line sent to the model is the `allocateBlock` line, from which it can only guess. For the other 3, the selected line is the anomalous event itself (a redundant `addStoredBlock`, an empty packet): line selection works when the anomaly is something that happened, not when it is something that did not.

Over the whole detection test period, the rules give a specific sentence to every one of the 1,744 flagged blocks: 1,034 writes never stored, 247 failed deletes, 167 replicas reported twice, 150 empty packets, 138 re-replications, 5 blocks of no file and 3 replication time-outs. 571 of them (33%) also name the Java exception the block logged. The sentences describe the trace in the log, not its origin (a crashed node and a network failure can leave the same trace), and the rules are specific to HDFS. Of the 138 re-replications, 79 are false alarms of the detector, since re-replication also happens in normal blocks.

4.7 Speed and cost

One request at a time, the small model takes about 3 seconds per explanation and costs more than the API, because the GPU is paid for while it waits (Table 6). With batches of 16 it is about 12 times faster and about 7 times cheaper than the teacher, with a peak GPU memory of 3.4 GB. Running locally pays off only when explanations can be grouped, which is the case when the detector flags many blocks at once. The GPU price covers the GPU alone, so the real gap is somewhat smaller.

5 Discussion

The input decides what can be explained. HDFS labels are per block and most anomalies are visible only at the block level: an event that is absent, or a rare event in another line. A model that sees one INFO line whose template also appears in normal blocks has no information about the cause. The teacher did not have it either when it wrote the training targets, so it guessed, and the student learned to reproduce the guess. Distillation transfers whatever the teacher writes, hallucinations included, and metrics computed against the teacher reward it for that.

Measuring against the teacher is not enough. ROUGE-L and BERTScore against the teacher rank the systems sensibly and show that fine-tuning works, yet they say nothing about correctness. The audit, which only needs the raw logs and the detector’s statistics, gives a very different picture. Any evaluation of generated explanations needs at least one check against the source data.

Cheap structure beats generation here. The list of missing events comes for free from the detector’s training data and explains 40% of the anomalous test blocks exactly; rare events cover the rest. On HDFS, a structured description of the block is a better explanation than a generated sentence about one line. A language model becomes useful again if its input is that structured description, so that the cause can actually be read from it.

Changing the input fixed it; the teacher is now the limit. v2 confirms the diagnosis: with the same model size and recipe, giving the block instead of one line takes the most common failure from 0% to 100%. What remains are the teacher’s mistakes, copied by the student. Better annotations (or a few corrected by hand) would help more than a bigger student. Rules and model are also complementary: the rules never miss the cause but call every re-replication a problem, the model is never fooled by re-replications but has blind spots.

Detection is easy where templates are stable. The logistic regression on template counts is close to perfect on HDFS and loses to a one-line rule on BGL. The difference is the template vocabulary, not the model.

6 Limitations and future work

- There are no expert root causes for HDFS. The audit checks explanations against block-level evidence with keywords and a 50-example manual check, not against expert labels.
- The detector is supervised and needs complete sessions; PCA works without labels but its threshold does not carry over from one period to the next.
- The training data for the explanation model is small and repetitive: 69 anomalous lines, 29 causes, 15 templates.
- Template extraction is a set of regular expressions designed for HDFS, which breaks down on BGL.

The next steps: correct the v2 annotations where the teacher is systematically wrong (normal deletions, empty packets), combine the rules with the model, and replace the regular expressions with a learned parser such as Drain so that detection holds on less regular systems.

7 Conclusion

A logistic regression on template counts detects HDFS anomalies almost perfectly, and a 1.5B-parameter model fine-tuned with LoRA reproduces the explanations of a 70B teacher better than

any baseline, at a fraction of the cost when requests are batched. But reproducing the teacher is not the same as explaining the anomaly: neither model finds the evidence, because it is not in the line they are given. Checking the explanations against the raw logs, rather than against the teacher, is what revealed it, and giving the model the whole block is what fixed it: the same small model then gets within five points of a 120B teacher.

References

- [1] W. Xu, L. Huang, A. Fox, D. Patterson, M. I. Jordan. Detecting large-scale system problems by mining console logs. *SOSP*, 2009.
- [2] S. He, J. Zhu, P. He, M. R. Lyu. Experience report: system log analysis for anomaly detection. *ISSRE*, 2016.
- [3] M. Du, F. Li, G. Zheng, V. Srikumar. DeepLog: anomaly detection and diagnosis from system logs through deep learning. *CCS*, 2017.
- [4] J. Zhu, S. He, P. He, J. Liu, M. R. Lyu. Loghub: a large collection of system log datasets for AI-driven log analytics. *ISSRE*, 2023.
- [5] A. Oliner, J. Stearley. What supercomputers say: a study of five system logs. *DSN*, 2007.
- [6] P. He, J. Zhu, Z. Zheng, M. R. Lyu. Drain: an online log parsing approach with fixed depth tree. *ICWS*, 2017.
- [7] Qwen Team. Qwen2.5 technical report. arXiv:2412.15115, 2024.
- [8] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. LoRA: low-rank adaptation of large language models. *ICLR*, 2022.
- [9] C.-Y. Lin. ROUGE: a package for automatic evaluation of summaries. *ACL Workshop on Text Summarization Branches Out*, 2004.
- [10] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, Y. Artzi. BERTScore: evaluating text generation with BERT. *ICLR*, 2020.

A Reproducing the results

```
python src/detect.py --log HDFS.log --labels anomaly_label.csv # Table 1
python src/detect_bgl.py --log BGL.log # Table 2
python src/train.py # LoRA training (GPU)
python src/evaluate.py # Table 3 (GPU)
python src/audit_explanations.py --log HDFS.log # Figure 4
python src/pipeline.py --log HDFS_window_3k.log --top 10 # Section 4.5
python src/benchmark.py # Table 4 (GPU)
python src/build_dataset_v2.py --log HDFS.log --labels anomaly_label.csv # v2 data
python src/annotate_v2.py --split train; python src/annotate_v2.py --split test
python src/train_v2.py; python src/eval_v2.py # Tables 5-6 (GPU)
python src/app.py --share # live demo (GPU)
```

B Example

A block whose write never completed (Loghub label: Anomaly). The pipeline output:

```
blk_5679332564404011413 score 1.000 2 lines
missing: PacketResponder <N> for block <BLK> terminating
missing: Received block <BLK> of size <N> from /<IP>
```

```
missing: BLOCK* NameSystem.addStoredBlock: blockMap updated: <IP> is added to <BLK>
size <N>
missing: Deleting block <BLK> file <PATH><BLK>
missing: BLOCK* NameSystem.delete: <BLK> is added to invalidSet of <IP>
line sent to the model:
081109 204403 35 INFO dfs.FSNamesystem: BLOCK* NameSystem.allocateBlock:
/user/root/rand/_temporary/_task_200811092030_0001_m_000337_0/part-00337.
blk_5679332564404011413
```

The fine-tuned model's answer for this kind of line:

```
{"cause": "Erreur d'allocation de bloc",
"raisonnement": "Étape 1 : Le système de fichiers HDFS tente d'allouer un bloc pour
un fichier
temporaire. Étape 2 : L'allocation du bloc échoue, ce qui indique une erreur dans
le processus
d'écriture de données. Étape 3 : ..."}

```

The allocation did not fail; the write that followed never completed.